ChatGPT

Autonomous Web-Application Self-Healing and Fault Diagnosis: Report

Executive Summary: Modern cloud-native web applications comprise many interacting services, making fault detection and recovery very challenging. Recent research emphasizes **multimodal telemetry** (metrics, logs, traces, events) and **AI/ML analysis** to localize and fix faults. A 2024 survey of 98 papers notes the field's rapid growth and the need for standardized benchmarks【9†L22-L28】. Advances like **DiagFusion** use embeddings and Graph Neural Networks to fuse metrics/logs/traces for root-cause localization 【18†L24-L33】. However, critical evaluations (e.g. *DiagMLP*) suggest simple ML baselines can rival complex GNNs, highlighting the need for better datasets【63†L58-L67】【63†L163-L172】. In practice, open-source AIOps platforms (e.g. **KubeHeal**, **Keep**) combine rule-based and ML logic with Kubernetes, Prometheus, and observability stacks to implement self-healing. Public datasets (Microsoft Azure traces【43†L271-L280】, Google Borg traces【46†L269-L277】, the AIOps Challenge data【33†L3388-L3396】, GAIA/TrainTicket logs【33†L3395-L3403】【35†L3422-L3430】, SWaT/WADI sensors【35†L3428-L3432】, Yahoo S5 anomaly series【40†L109-L117】, etc.) can train and benchmark fault detectors. We outline recommended ML architectures (from simple threshold rules and decision trees to LSTM/Transformer and GNN models) with features from parsed logs, time-series metrics, and traces. Implementation best practices include Spring Boot/Node.js microservices on Kubernetes, instrumented by Prometheus/Grafana and OpenTelemetry, pipelined with CI/CD (ArgoCD/Tekton) and chaos tools (Chaos Mesh/Litmus). Experimental design should test multiple fault scenarios (CPU spike, memory leak, pod crash, network partition, etc.) with repeated injections and statistical analysis of recovery rates and detection latencies. Key challenges remain in reliable labeling, explainability, and human–machine safety.

1. Literature Survey (2018–2026): Research spans academic venues, whitepapers, and open-source projects. A comprehensive survey by Zhang *et al.* (2024) categorizes 98 studies on microservice fault diagnosis【9†L22-L28】. Key approaches include **rule- or heuristic-based** (threshold alarms, dependency graphs), **supervised learning** on labeled failures, **anomaly detection** (unsupervised ML), and **multimodal fusion** using deep models. For example, many works use **log parsing** (e.g. Drain3) to convert raw text into template counts, align with metrics, and build service graphs【63†L163-L172】. CNN/LSTM/Transformer networks model temporal metrics, while language models embed logs【63†L169-L172】. **Graph Neural Networks** (GNNs) have become popular to integrate logs/metrics with the call-graph, but recent work (DiagMLP) finds that a simple MLP on preprocessed features can match GNN accuracy, calling into question the complexity needed【63†L58-L67】.

Open-source industry tools and whitepapers (e.g. IBM CloudPak, Kubernetes/SRE blogs, CNCF AIOps) also inform best practices. Notable systems include **IBM Watson AIOps**, **Azure Monitor AIOps**, and open-source stacks like **Prometheus/Grafana**. The "*Awesome Failure Diagnosis*" list【11†L447-L459】 compiles relevant toolkits: Prometheus (with node-exporter, Blackbox exporter, etc.) for metrics【11†L453-L460】; ELK/EFK or Grafana Loki for logs【11†L463-L470】; Jaeger/Zipkin or SkyWalking with OpenTelemetry for traces【11†L471-L475】. Embedding/explainability is addressed via attention and SHAP in some recent DL works【63†L169-L172】. Strengths of surveyed methods include automated fusion of heterogeneous data

and end-to-end learning, while limitations include reliance on synthetic datasets, limited generalization, and scarce labeled real failures【9†L22-L28】【63†L58-L67】.

2. Open-Source AIOps/Self-Healing Systems: We catalog key systems and tools:

- **KubeHeal (GitHub)**: An OpenShift/Kubernetes-based self-healing platform. It combines a *deterministic layer* (config operators, rule-based actions) and an *AI-driven layer* (anomaly detection via Isolation Forest/LSTM, root-cause analysis) under a coordinating Flask engine【30†L844-L853】【30†L857-L861】. It uses a shared observability stack (Prometheus + AlertManager) and can roll out remediations (restarts, scaling) via the K8s API【30†L856-L864】. KubeHeal's hybrid design and CI/CD (ArgoCD, Tekton) make it a production-grade reference【30†L844-L853】.
- **Keep (GitHub)**: A generic open-source AIOps/workflow engine. Workflows (YAML) are triggered by alerts (e.g. from Sentry or Prometheus) and can execute actions like sending Slack messages, creating Jira tickets, or even restarting pods【57†L440-L449】. It functions like "GitHub Actions for monitoring", integrating with many notification and incident tools【57†L440-L449】. While not specialized to fault localization, Keep illustrates an alert-driven automation pattern.
- **Prometheus + Alertmanager**: Core monitoring stack. It continuously scrapes metrics (via exporters) and fires alerts (thresholds or anomaly rules). Alertmanager can route alerts and invoke webhooks or Kafka messages. This integration is common in Kubernetes: for example, many operators watch Prometheus alerts to trigger recovery flows.
- **OpenTelemetry/Jaeger/SkyWalking**: For tracing and logs. These collect distributed traces (Spans) and structured logs, which can feed ML analyzers. Projects like SkyWalking even offer built-in diagnostics.
- **Chaos Mesh / LitmusChaos / Pumba**: Though focused on injection, these are part of the ecosystem. They provide CRDs or tools to inject faults (pod kill, CPU stress, network cut).
- **Operator-Based Tools**: Some operators (Kubernetes controllers) encapsulate self-healing, e.g. custom Reconcilers that check health and reconcile states. The OperatorHub (see **KubeHeal**, **Robusta**, etc.) is a registry for such solutions.

The table below compares these on data modalities, actions, safety, and explainability. (✓ = supported, – = not applicable):

System/Tool	Metrics	Logs	Traces	Recovery Actions	Safety-Gates	Explainability
KubeHeal (OpenShift) 【30†L844-L853】【30†L857-L861】	✓	✓	✓	Pod restart, config rollout, scaling via K8s API	Priority conflict resolution (coordination engine) 【30†L852-L860】	ML models opaque (partial)

System/Tool	Metrics	Logs	Traces	Recovery Actions	Safety-Gates	Explainability
Keep (GitHub)【57†L440-L449】	✓ (via alerts)	✓	–	Create tickets, Slack msgs, invoke Webhooks (can restart)	User-defined workflow conditions (if-clauses)【57†L469-L478】	Declarative logic (YAML)
Prometheus/AlertMgr【11†L453-L460】	✓	–	–	Alerts only; often integrated with operators or external scripts	Grouping/ dedup in AlertMgr	N/A (threshold rules)
OpenTelemetry【11†L471-L475】	–	✓ (structured logs)	✓	n/a (observability only)	n/a	(supports context propagation)
Grafana Loki/EFK【11†L463-L470】	–	✓	–	n/a (log store; can feed alerts)	n/a	(templates)
Jaeger / Zipkin【11†L471-L475】	–	–	✓	n/a (trace store)	n/a	(visual)
Chaos Mesh / Litmus / Pumba	–	–	–	Fault injection (dist. across pods)	Manual orchestration	n/a
Custom K8s Operator	✓/✓	✓	✓	Custom CRD-driven actions	Defined in code	(depends on design)

Table: Comparison of open-source/self-healing systems.

3. AI/ML Architecture Recommendations: We outline a spectrum of approaches for fault detection/ localization using metrics, logs, and traces:

- **Rule-Based Baselines:** Simple threshold alarms on CPU/memory, or rule-pattern matching on logs (regex/KQL). Fast to deploy but high false positives and no learning. Useful for baseline or safety filters (e.g. a “circuit breaker” that triggers if CPU>90% for 5min).
- **Classical ML (RF, XGBoost, SVM):** With labeled incidents, one can engineer features (moving averages, rates, error counts) and train tree ensembles to classify fault types or predict anomalies. For example, histogram features from metrics plus one-hot log-templates can serve as input to

XGBoost. Random forests offer quick training and built-in feature importance. These require a labeled dataset of normal vs faulty, and often achieve moderate accuracy.

- **Anomaly Detection (Isolation Forest, One-Class SVM, Autoencoders):** Unsupervised methods that model "normal" behavior on metrics/log features and flag outliers. Common in AIOps due to scarce labels. E.g. Isolation Forest on a sliding-window of CPU, latency, and error-rate features. Autoencoder neural nets can learn to reconstruct normal patterns, flagging reconstruction error spikes as anomalies. These are used as an ML layer for initial alarm generation.
- **Sequence Models (RNN/LSTM/Transformer):** Deep learning on time-series for anomaly prediction or sequence labeling. For instance, an LSTM-autoencoder trained on historical metric windows can detect anomalies by high prediction error. Transformers (with attention) can capture complex temporal dependencies and handle irregular intervals. For logs, sequence models (LSTM or Transformer) can treat log-event sequences as text (or embed templates with word2vec/BERT) to predict next-event or classify.
- **Graph-Based Models (GNNs):** To leverage service topology, one can build a **dependency graph** (nodes=service instances, edges=inferred from traces or service config) and embed each node with its telemetry vector. A GNN (e.g. GraphSAGE, GAT) can then propagate anomalies through the graph and output per-node fault likelihoods. DiagFusion is one such example, where a GNN over a service dependency graph localizes the root cause【18†L24-L33】. However, note that recent experiments found that a flat MLP on the same embeddings (ignoring graph structure) performed similarly【63†L58-L67】. Thus, complex GNNs should be justified by improved performance.

For all models, **input features** come from preprocessing telemetry: metrics are normalized time-series (possibly downsampled or aggregated), logs are parsed into template counts or embedded vectors, traces yield latencies or call graphs. Feature engineering may include sliding-window statistics (mean, variance), derivatives, or one-hot encodings of error keywords. Preprocessing is critical: e.g. using Drain3 to parse raw logs into templates, then counting occurrences per time interval【63†L163-L172】; normalizing metrics per service; stamping events. Models can use *multimodal fusion*: for example, concatenate log, metric, and trace embeddings for each service instance.

Model training: Depending on label availability, use supervised loss (cross-entropy for classification or multi-label fault types), semi-supervised (autoencoder anomaly loss), or unsupervised (one-class). Time-based train/test splits (to respect temporal ordering) are essential. Common strategies include random splits of fault injection runs, or leave-one-fault-type-out validation. Over time, models should be retrained as new data arrives (continuous learning) to adapt to changes.

Evaluation Metrics: Go beyond raw accuracy. For fault localization, use *Top-K accuracy* (whether true root cause is among the top K outputs) and Mean Reciprocal Rank (MRR)【67†L7-L10】, since we often care if the actual faulty service is highly ranked. For anomaly detection, use precision/recall (or F1) on anomaly labels. Also measure *detection latency* (how long after a fault begins before alarm or fix) and *MTTR* (mean time to recovery). In ML comparisons, use standard metrics: confusion matrices, AUC for binary anomalies, etc.

Explainability: Deploy SHAP or attention analyses to highlight which features or logs drove a prediction. For tree models, feature importance is built-in. For sequence models, attention weights can indicate critical time points. Some works build "evidence graphs" linking symptoms (e.g. metric spikes, error log lines) to causes【63†L161-L170】. Interactive dashboards (Grafana) can also help SREs drill into anomalies.

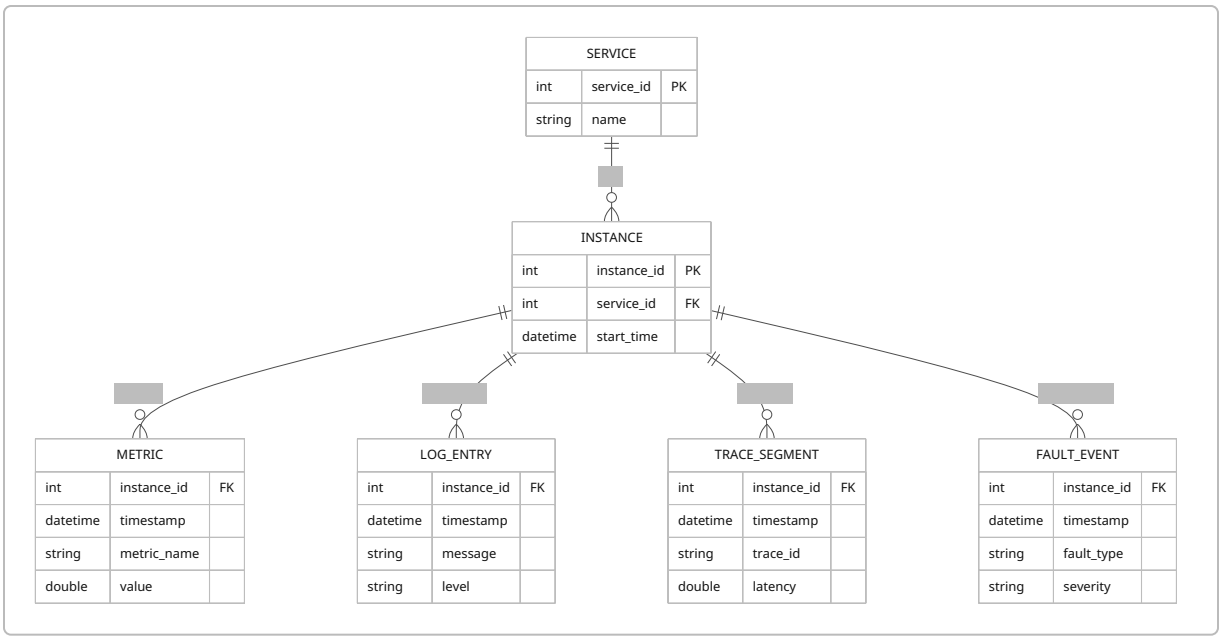
4. Dataset Inventory: Important public data sources include:

- **Microsoft Azure (AzurePublicDataset)**【43†L271-L280】: VM and function traces. Contains VM workload traces (2017, 2019) and serverless invocation logs (Azure Functions 2019–2020). These contain timestamps, resource usage, request logs, and can be used to simulate cloud faults 【43†L276-L284】.
- **Google Borg Cluster Data (ClusterData2019/2011)**【46†L269-L277】: Large-scale traces from Google's data centers. *ClusterData2019* has data from 8 clusters over a month, *ClusterData2011* has a 12.5k-node trace. They include per-machine CPU/mem usage and job events, useful for anomaly detection though they lack explicit fault labels【46†L269-L277】.
- **AIOps Challenge Datasets (2020–2022)**【33†L3388-L3396】: Collected by Tsinghua/China Mobile from production microservices (finance, e-commerce). Provide **metrics (M)**, **logs (L)**, **traces (T)** and **topology (TP)** of real systems. For example, the 2020 challenge data is from China Mobile Zhejiang, and 2021/2022 data from banking and e-commerce systems【33†L3388-L3396】.
- **GAIA (Generic AIOps Atlas)**【33†L3395-L3403】: A synthetic microservice dataset (CloudWisdom's MicroSS). Contains ~6,500 metrics, 7M log entries, and full traces over two weeks, with injected failures and topology. Very rich and anonymized, meant for root-cause analysis research【33†L3395-L3403】.
- **TrainTicket (DejaVu, Eadro, Nezha)**【35†L3422-L3430】: Fudan University's microservice benchmark logs. Each variant includes service logs, metrics, and traces with simulated fault injections. Widely used in academic experiments【35†L3422-L3430】.
- **Loghub**: A Chinese log corpus aggregating diverse open-source logs (77 GB total) across operating systems, clusters, applications, etc.【35†L3428-L3432】. Useful for log-specific models.
- **SWaT / WADI**【35†L3428-L3432】: Industrial sensor datasets from a water treatment testbed (SWaT) and distribution network (WADI). Provide multivariate time-series with labeled attack anomalies on physical processes.
- **NASA SMAP / MSL**【35†L3430-L3432】: Satellite telemetry data from soil moisture (SMAP) and Mars rover (MSL) missions, with labeled anomalies in sensor streams. Common benchmarks for multivariate anomaly detection.
- **Yahoo S5 (YData Anomalies)**【40†L109-L117】: A standard time-series anomaly dataset with 371 series (real and synthetic), each with an `is_anomaly` flag per timestamp. Files have fields `timestamp,value,is_anomaly` for real (A1) and synthetic (A2–A4) series【40†L109-L117】.
- **Server Machine Dataset (SMD)**【52†L429-L437】: Telemetry from enterprise servers (38-dimensional metrics from 5 machines). Used for unsupervised anomaly detection (OmniAnomaly benchmark)【52†L429-L437】.
- **Microsoft Azure Function Dataset (2019–2020)**: Invocation traces for serverless workloads (from [43†L279-L288]).
- **Yahoo Blogs/HIPerShop/SockShop**: Not real data, but these open-source microservice apps (HipsterShop, Sock Shop) are used to generate synthetic datasets via chaos testing. For instance, experiments often deploy HipsterShop on K8s and inject faults to create bespoke logs/metrics 【35†L3477-L3484】.

Synthetic Data & Fault Injection: In addition to public traces, synthetic generation is common. Fault-injection tools (Netflix **Chaos Monkey**, CNCF **LitmusChaos**, **Chaos Mesh**, **Pumba** for Docker) can programmatically introduce CPU hogs, pod restarts, network delays, etc. These produce labeled traces (the exact injection time is known), which are crucial since real-world labels are scarce. One approach is to simulate a microservice workload (e.g. load testing Sock Shop) and use chaos tools to trigger known faults,

capturing all telemetry. Data augmentation can include adding noise, varying fault timings, or mixing multiple concurrent faults. Labeling strategy: any period affected by injection is anomalous; root-cause label is the injected service. In semi-supervised schemes, one may label just end-of-spectrum anomalies.

A **recommended data schema** for telemetry might be:



This schema (visualized above) shows how **Service** → **Instance**, with instances having many metric, log, trace, and fault records. Each Metric/Log/Trace has a timestamp and content. This unified schema supports building ML datasets (one can join metrics/logs by timestamp) and labeling faults per instance.

5. Implementation Patterns & Tech Stack: A production prototype would typically use microservices written in **Spring Boot and Node.js** (as specified) running on **Kubernetes**. Observability is built with **Prometheus** for metrics (scraping /metrics endpoints and system exporters【11†L453-L460】), **Grafana** for dashboards, and **OpenTelemetry** agents to collect traces/logs into **Jaeger/SkyWalking** or **ELK/Loki** stacks 【11†L463-L470】【11†L471-L475】. **Kafka** can be used as a centralized event bus for alerts or log streaming.

For CI/CD, tools like **Tekton Pipelines**, **ArgoCD**, or **Jenkins/GitOps** can automate build-tests-deploy. Each commit triggers unit and integration tests, then container image builds, and a deployment (rolling update via Helm charts/Operators).

Fault Injection: Integrate chaos tools into testing pipelines. E.g. use **Chaos Mesh** CRDs to define network partitions or pod CPU limits; or run **LitmusChaos** experiments before production pushes. For lighter tests, **Pumba** can randomly kill containers or throttle traffic on demand. These injectors should be scriptable (e.g. via a Kubernetes Job or GitOps workflow) and have clean-up steps.

Recovery Execution: Automated remediations can be implemented via Kubernetes API calls, Helm rollbacks, or custom operators. For example, a recovery action could be "`kubectl rollout restart`"

for a Deployment, or applying a stable Helm chart revision. Custom operators (possibly written with the Fabric8 or Operator SDK) can encapsulate healing logic: e.g. an Operator watches alerts and scales or heals the cluster. SREs should define *safety gates* – conditions under which automation stops (e.g. if CPU>95% still after 5min, pause for human review).

6. Experimental Design & Benchmarks: To evaluate the system, design controlled experiments with multiple **fault scenarios**: CPU spike, memory leak, service crash, service hang, disk fill, network partition, and combinations. Also vary workload patterns (steady load vs traffic spikes). Each scenario should be run **N** times (e.g. 30–50 runs) to gather statistically significant results. Use randomization (different seed timings for faults) to avoid bias.

Record metrics/logs/traces continuously. Define **success criteria** such as "Root cause identified in top-1 output" or "service recovered (healthy) within T seconds". Statistical tests (e.g. t-test or Mann-Whitney) can compare variants (with AI vs baseline rules) on metrics like detection precision, recall, MRR, and recovery time. Report confidence intervals (±std error) for all key metrics. Example: measure the time from fault injection to alarm (detection latency), from alarm to action (orchestrator response time), and from action to recovery (true fix). Set **safety thresholds**: e.g. if mean recovery time > 60s or detection F1 < 0.8, consider the approach unstable.

7. Risks, Challenges & Gaps: Key challenges include: (a) **Data scarcity & labeling**. High-quality, labeled fault data in real production is rare. Many studies rely on simulations, which may not reflect complex environments. (b) **Generalization:** Models trained in one environment may not transfer to another (different app, workload). (c) **Explainability & Trust:** Operators may not trust black-box fixes; lacking clear explanations for "why" a fix was applied is problematic. (d) **Conflicting Actions:** Autonomous actions might conflict (e.g. simultaneous scaling and restart), so coordination/safety gates are critical (like KubeHeal's conflict resolution)【30†L851-L860】. (e) **Overhead:** Continuous ML inference and data collection can add overhead and noise. (f) **Adversarial Faults:** Rare or multi-service faults may evade detection. (g) **Novelty Detection:** New fault types (e.g. from third-party libraries) may not be recognized. (h) **Regulation & Security:** Automatic remediation must avoid cascading failures; one must guard against "runaway automation."

Research gaps include standardized **benchmarks** for cross-comparison (the field lacks agreed-upon evaluation scenarios). Also, most work focuses on root-cause localization, not on **action selection**; combining diagnosis with safe automated fixes remains open. There is opportunity in **LLM-based reasoning** for AIOps (few publications yet, e.g. IBM's Agentic Ops concept), and in **digital twin** simulators that can safely test healing policies. Finally, integrating **causal reasoning** and knowledge graphs to explain failures is an active area (beyond our project scope).

8. Key References and Resources: (Annotated reading list, ordered by relevance)

- *Zhang et al.*, 2024 – *"Failure Diagnosis in Microservice Systems: A Comprehensive Survey"*【9†L22-L28】. An extensive survey of 98 papers (2003–2024) on microservice failure diagnosis. Summarizes modalities, methods (statistical, ML, AI), and provides a catalog of datasets and toolkits. Essential background reading.
- *Zhang et al.*, 2023 – *"DiagFusion: Robust Failure Diagnosis via Multimodal Data"*【18†L24-L33】. Proposes a GNN-based method combining metrics, logs, and traces for root-cause localization.

Introduces embedding/data augmentation techniques and reports large accuracy gains over single-modal approaches. Source code: GitHub DiagFusion.

- *Zhang et al.*, 2025 – *"Are GNNs Effective for Multimodal Fault Diagnosis in Microservice Systems?"*【63†L58-L67】. Critically evaluates the GNN paradigm by comparing it to a simple MLP (DiagMLP) on five public datasets. Finds the MLP baseline often matches or exceeds GNNs, highlighting the need for better datasets and evaluations. Discusses common preprocessing (log parsing, clustering, etc.)【63†L163-L172】.
- *Pham et al.*, 2024 – *"RCAEval: A Benchmark for Root Cause Analysis"*【65†L154-L162】. Introduces a suite of labeled microservice traces for evaluating RCA algorithms (ASE'24, WWW'25, FSE'26). Official repository: RCAEval GitHub. A key resource for testing fault localization models.
- *Keep (keephq), 2023 –"Keep: Open-Source AIOps Platform"*【57†L440-L449】. GitHub project for alert-driven automation workflows. Each YAML workflow defines triggers (alerts), steps (data enrichment), and actions (notifications, ticket creation, service restarts). Good example of declarative incident remediation.
- *KubeHeal (Red Hat), 2023 –"OpenShift AIops Platform"*【30†L844-L853】【30†L857-L861】. GitHub repository of a Kubernetes-based self-healing system. Illustrative architecture (Flask coordination, CI/CD with Tekton, Helm charts) and hybrid rule/ML design. See its 📊 *Architecture* section for the layered approach.
- **Microsoft Azure Public Dataset, 2023 – "Azure Traces"**【43†L271-L280】. Official Azure dataset repository. Contains VM workload traces and serverless invocation logs (2017–2024) under CC-BY-4.0. Useful for realistic cloud telemetry. (Download links in repo.)
- **Google Cluster Data, 2019 – "Borg Cluster Traces"**【46†L269-L277】. Open data from Google: *ClusterData2019* (8 cells, May 2019) and *ClusterData2011* (12.5k nodes, May 2011). Contains machine-level metrics (CPU, RAM) and scheduling events. License: CC-BY. Good for anomaly analysis research.
- **Yahoo Research, 2023 – *"YData Labeled Time-Series Anomalies (v1.0)"***【40†L109-L117】. The Yahoo S5 anomaly detection dataset. Describes the A1–A4 benchmarks (real and synthetic series) and fields (`timestamp,value,is_anomaly`). Hosts 371 series for evaluating time-series anomaly detectors.
- **Hennhoefer, 2023 – *"Awesome Time-Series Anomaly Datasets"***【52†L429-L437】. GitHub curated list of public TS datasets. Includes *Server Machine Dataset (SMD)*【52†L429-L437】 (5 machines, 38 sensors, used by OmniAnomaly), and many others (KPI/AIOps metrics, NAB, etc.). Excellent index of benchmarks.
- **Prometheus Docs (official)**【11†L453-L460】. Documentation and exporter guides for Prometheus. Covers using node_exporter, cadvisor, blackbox exporter, and TSfresh feature extraction. Cited for instrumentation best practices.
- **Grafana Loki Documentation**【11†L463-L470】. Guides on deploying Loki (logs) and integration with Grafana/Promtail. Part of the ELK/Loki stack. Useful for log collection and exploration.
- **OpenTelemetry/SkyWalking/Jaeger**【11†L471-L475】. References to distributed tracing projects. SkyWalking and Jaeger are widely used; OTEL provides instrumentation for Spring Boot/Node.js. Good for learning tracing integration.
- **SWaT/WADI Datasets (2016)**【35†L3428-L3432】. Public water treatment plant sensor data with injected attack anomalies. Available from SUTD labs. Often cited in anomaly-detection literature.
- **NASA SMAP/MSL Datasets (2016)**【35†L3430-L3432】. Satellite sensor telemetry with labeled anomalies. Provided by NASA (open access). Canonical multivariate anomaly-detection benchmark.
- **LogHub (2018)**【35†L3428-L3432】. A collection of diverse log datasets compiled by HKU. Contains system, cluster, web, and app logs (77+GB). Useful for training log parsing and anomaly methods.

- **Nedelkoski *et al., 2020 – "Multi-Source Distributed System Data for AI-Powered Analytics"***. Conference paper discussing the benefits of multimodal (logs+metrics) datasets in cloud systems (ESOCC 2020). Argues joint analysis boosts detection of complex faults.
- **Mariani *et al., 2018 – "Localizing Faults in Cloud Systems" (ICST 2018)***. Presents one of the first log-based fault localization techniques, using statistical analysis of log clusters. A baseline classical approach.
- **DeepLog (ICML 2017)**. An LSTM-based method for log anomaly detection (Du *et al.*). Introduces parsing logs to templates then training a model to predict next log event. Widely cited early DL approach.
- **Isolation Forest (Liu *et al.*, 2008)**. Foundational anomaly detection algorithm. Conceptual reference for unsupervised faults on metric data.
- **SHAP (Lundberg & Lee, 2017)**. Not specific to AIOps, but the go-to method for feature-level explainability in ML. Useful for explaining fault classifiers.

Other resources: The OperatorHub registry lists community Kubernetes operators (search "self-healing" or "AIOps"). CNCF webinars and GitHub (e.g. Chaos Mesh, LitmusChaos) provide documentation and diagrams for chaos testing. For CI/CD patterns, see the Argo Rollouts and Tekton examples.

Each of these references is a starting point to deeper details on methods, datasets, or tools mentioned above.
